Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Aynı günün teorik dersi ders 9 kaydındadır. Bu kayıt onun uygulamasıdır.
  - Kaydın ilk 4 dakikası öğrencilerle Veri Yapıları dersindeki ödev üzerine sohbettir.
- Akbil (İstanbulkart) sistemi örneği NetBeans'te canlı kodlandı.
  - `Person` üst sınıfı, `Student` ve `Teacher` alt sınıfları, sonra `Emekli` sınıfı.
  - `extends`, `super(...)`, yapıcı aşırı yüklemesi (constructor overloading).
  - `bakiyeGuncelle()` metodunun override edilmesi ve polymorphism.
  - `toString()` override'ı.
  - Üst sınıf tipinde referans dizisi (upcasting) ile tüm kişileri tek döngüde işlemek.
  - Nesne oluşturulurken yapıcıların çalışma sırası ve örtük `super()`.
  - `super.metot()` ile üst sınıftaki metodu çağırmak.
  - Abstract metot ve abstract sınıfa giriş.
- Alıştırma önerisi: Örneğe okul sınıfı ekleyip association kurmak.

Hocanın Özellikle Vurguladığı Kısımlar

- Kalıtım ve polymorphism sayesinde `if`/`switch` zincirine gerek kalmaz.
  - Öğrenci mi öğretmen mi diye `if` ile bakmak yerine her alt sınıf kendi `bakiyeGuncelle()` metodunu yazar.
  - Yeni bir kategori (ör. emekli) eklemek için mevcut sınıflara dokunmadan yeni bir alt sınıf yazmak yeterlidir.
- Üst sınıf tipinde referans dizisi, farklı alt sınıf nesnelerini tek döngüde işlemeyi sağlar.
  - Ör. 600 kişinin hepsine 5 lira bonus yüklemek tek döngüyle yapılır.
- Alanları `private` tutun. Alt sınıftan bile get/set metotlarıyla erişin.
- "Abstract sınıfın en az bir abstract metodu olmalıdır" önermesi **yanlıştır**. Tersi doğrudur: Bir sınıfta en az bir abstract metot varsa sınıf abstract olmak zorundadır.
- Abstract sınıftan nesne oluşturulamaz.
- Abstract metot bir standart koyar. Alt sınıflar bu metodu yazmak zorundadır. Hocanın benzetmesi: Avrupa Birliği'ne girmek için yerine getirilmesi gereken şartlar.
- Bol kod yazın. Slayttaki örneklerin kodlarını tekrar amaçlı inceleyin.

Kısa Tekrar Notları

- `class Student extends Person` ile kalıtım kurulur.
- Alt sınıfın yapıcısı, üst sınıfın alanlarını `super(id, yas, bakiye)` ile doldurur.
- `super(...)` yazılmazsa üst sınıfın parametresiz yapıcısı örtük olarak çağrılır. Üst sınıfta parametresiz yapıcı yoksa derleme hatası alınır.
- Nesne oluşturma sırası: önce üst sınıfın yapıcısı, sonra alt sınıfın yapıcısı.
- `@Override` notasyonu zorunlu değildir. Aynı imzalı metot yazıldığında NetBeans bunu eklemeyi önerir (uyarı, hata değil).
- `Person p = new Student(...);` geçerlidir (upcasting).
- `Student s = new Person(...);` geçersizdir: `incompatible types: Person cannot be converted to Student`.
- `super.bakiyeGuncelle()` ile override edilen metodun üst sınıftaki hâli çağrılabilir.
- `public abstract double bakiyeGuncelle();` gövdesizdir ve `;` ile biter.

- Abstract metodu override etmeyen alt sınıf derleme hatası verir: `Student is not abstract and does not override abstract method bakiyeGuncelle()`.
- `new Person()` (Person abstract iken): `Person is abstract; cannot be instantiated`.
- `final class Person` yazılırsa `Person`'dan kalıtım alınamaz.

Detaylı Açıklamalar

Akbil sistemi: sınıflar

- Her kullanıcının ID, yaş ve bakiye bilgisi var. Bunlar ortak olduğu için `Person` üst sınıfına konur.
- Yaş güncelleme herkes için aynıdır. Bu yüzden `yasGuncelle()` yalnızca `Person`'da yazılır.
- Bakiye güncelleme kullanıcı türüne göre değişir: öğrenciden 0,5 TL, öğretmenden 1 TL düşülür. Bu yüzden alt sınıflarda override edilir.
- `Teacher` sınıfına `tecrube`, `Student` sınıfına `bolumAdi` alanı eklendi.

Derste yazılan kodun derlenip çalıştırılan bir özeti (karelerden ve anlatımdan; Java ile denendi):

```
public abstract class Person {
    private int ID;
    private int yas;
    private double bakiye;

    public Person() {
        System.out.println("Person oluşturuldu");
    }
    public Person(int ID, int yas, double bakiye) {
        this.ID = ID;
        this.yas = yas;
        this.bakiye = bakiye;
    }
    public int getID() { return ID; }
    public int getYas() { return yas; }
    public double getBakiye() { return bakiye; }
    public void setBakiye(double bakiye) { this.bakiye = bakiye; }

    public int yasGuncelle() {
        this.yas++;
        return this.yas;
    }

    public abstract double bakiyeGuncelle();
}

public class Student extends Person {
    private String bolumAdi;

    public Student(int ID, int yas, double bakiye, String bolumAdi) {
        super(ID, yas, bakiye);
        this.bolumAdi = bolumAdi;
    }

    @Override
    public double bakiyeGuncelle() {
        this.setBakiye(this.getBakiye() - 0.5);
        return this.getBakiye();
    }

    @Override
    public String toString() {
```

```
        return "[ID=" + this.getID() + ", YAS=" + this.getYas()
             + ", BAKIYE=" + this.getBakiye() + ", BOLUM=" + this.bolumAdi + ".]";
    }
}

public class Teacher extends Person {
    private int tecrube;

    public Teacher(int ID, int yas, double bakiye, int tecrube) {
        super(ID, yas, bakiye);
        this.tecrube = tecrube;
    }

    @Override
    public double bakiyeGuncelle() {
        this.setBakiye(this.getBakiye() - 1.0);
        return this.getBakiye();
    }

    @Override
    public String toString() {
        return "[ID=" + this.getID() + ", YAS=" + this.getYas()
             + ", BAKIYE=" + this.getBakiye() + ", TECRUBE=" + this.tecrube + ".]";
    }
}
```

- `Student` içinde `this.ID` yazılamaz, çünkü `ID` `Person`'da private. `getID()` kullanılır.
  - Hoca, alanı `protected` yapınca `this.ID`'ye erişilebildiğini gösterdi. Ama get/set varken buna gerek görmedi.
- Hoca önce `bakiyeGuncelle()` içinde `return 0.0;` yazdı. Yazdırılan "yeni bakiye" 0.0 çıkınca `return this.getBakiye();` olarak düzeltti.

Yapıcıların seçilmesi

- `Teacher` sınıfına yapıcı yazılmadan `extends Person` denince hata alındı: `constructor Person in class Person cannot be applied to given types.`
  - Sebep: Alt sınıfın örtük yapıcısı, üst sınıfın parametresiz yapıcısını arar. `Person`'da yalnızca parametreli yapıcı vardı.
- NetBeans'te `Insert Code > Constructor` ile üst sınıfın hangi yapıcısının kullanılacağı seçilebilir.
- Hangi üst sınıf yapıcısının çağrılacağı, argümanların tipine ve sırasına göre belirlenir. Belirsiz bir durum olursa derleyici hata verir.

Polymorphism ve üst sınıf tipinde dizi

```
Person[] kisiler = new Person[6];
kisiler[0] = new Student(1, 15, 15.0, "Bilgisayar");
kisiler[1] = new Teacher(2, 40, 15.0, 20);
kisiler[2] = new Teacher(3, 55, 25.0, 20);
kisiler[3] = new Student(4, 22, 30.0, "Elektronik");
// kisiler[4] = p;  (aşağıdaki p nesnesi, 00:41:19)

int i = 1;
for (Person kisi : kisiler) {
    kisi.yasGuncelle();      // herkes için Person'daki metot
    kisi.bakiyeGuncelle();   // nesnenin gerçek sınıfındaki metot
    System.out.println(i + "---" + kisi);   // toString() çağrılır
    System.out.println(i++ + "---" + kisi);
}
```

- `new Person[6]` 6 nesne değil, 6 **referans** oluşturur. Her konuma ayrıca `new` ile nesne atanmalıdır.
- Döngüde `kisi` öğrenciyse `Student`'taki, öğretmense `Teacher`'daki `bakiyeGuncelle()` çalışır.
- `System.out.println(kisi)` yazmak, nesnenin `toString()` metodunu çağırır.
- Derste dizinin son konumları boş kaldığı için çıktının sonunda `NullPointerException` alındı. Hoca bunu exception konusunda ele alacaklarını söyledi.
- Bir öğrenci sordu: Dizinin elemanları sabit boyutlu olmalı, ama `Student` ve `Teacher` nesnelerinin boyutu farklı değil mi?
  - Cevap: Dizi nesnelerin kendisini değil, **referanslarını** (adreslerini) saklar. Referansların boyutu aynıdır.

Statü değiştirme: öğrenciyi öğretmen yapmak

```
Person p = new Student(5, 15, 10.0, "Bilgisayar");
p = new Teacher(p.getID(), p.getYas(), p.getBakiye(), 0);
```

- `p` üst sınıf tipinde olduğu için önce öğrenci, sonra öğretmen nesnesini gösterebilir.
- Yeni nesne oluşturulurken eski nesnenin ortak bilgileri (ID, yaş, bakiye) aktarılır. Bölüm bilgisi kaybolur.
- `p` `Student` tipinde tanımlansaydı `Teacher` nesnesi atanamazdı. `Student` ve `Teacher` aynı düzeyde, birbirine dönüştürülemeyen sınıflardır.
- Ters yönde atama yapılamaz: `Student pp = new Person(...);` derleme hatası verir. Hoca bunu `long`'un `int`'e kendiliğinden dönüştürülememesine benzetti.

Yapıcıların çalışma sırası

- `Person` ve `Student` sınıflarına parametresiz yapıcılar eklendi:

```
public Person() {
    System.out.println("Person oluşturuldu");
}

public Student() {
    System.out.println("Öğrenci oluşturuldu");
}
```

- `new Student()` çalıştırılınca çıktı:

```
Person oluşturuldu
Öğrenci oluşturuldu
```

- `Student()` içinde `super()` yazılmasa da derleyici gizli bir `super()` ekler.
- Hocanın anlattığı sıra: önce static üyeler, sonra üye alanlar (member variables) hazırlanır, sonra yapıcı gövdesi çalışır. Bu işlem önce üst sınıf, sonra alt sınıf için yapılır.

`super.metot()` ile üst sınıftaki metodu çağırmak

- `Person`'daki `bakiyeGuncelle()` başlangıçta "Person bakiye güncelleniyor" yazıyordu. Alt sınıflar override ettiği için bu satır hiç görünmedi.
- `Student`'taki metodun başına `super.bakiyeGuncelle();` eklenince önce "Person bakiye güncelleniyor", sonra "Std bakiye güncelleniyor" yazdı.
- Kurgu gerektiriyorsa ortak işlem üst sınıfta, özel işlem alt sınıfta yapılabilir.
- Metot içindeki `super.metot()` çağrısının ilk satırda olma zorunluluğu yoktur. Bu kural yalnızca yapıcıdaki `super(...)` içindir.

Abstract metot ve abstract sınıf

- `Person`'daki `bakiyeGuncelle()` hiçbir işe yaramıyorsa gövdesi boş olsun istenir. Ama `double` dönen bir metotta `return` yazılmazsa hata alınır.
- Çözüm: metodu `abstract` yapmak.

```
public abstract double bakiyeGuncelle();
```

- Abstract metot gövdesizdir. Bir sınıfta en az bir abstract metot varsa sınıf da `abstract` olmalıdır: `public abstract class Person`.
- Hoca, abstract metodu olmayan bir sınıfı `abstract` yaptı. Hata vermedi. Yani abstract sınıfın abstract metodu olmak zorunda değildir.
- Abstract metodu yazmayan alt sınıf hata verir. NetBeans "Implement all abstract methods" önerisiyle metodu otomatik ekler.
- `Person` abstract olunca `new Person()` hata verir: `Person is abstract; cannot be instantiated`.
- `Person p = new Student(...)` gibi üst sınıf tipinde referans kullanmak yine serbesttir.
- Bir öğrenci sordu: `bakiyeGuncelle()` üst sınıftan tamamen silinirse ne olur?
  - Alt sınıflarda yine yazılabilir. Ama `Person` tipindeki `kisi` üzerinden `kisi.bakiyeGuncelle()` çağrılamaz, polymorphism kaybolur.
- Abstract sınıfların teorik ayrıntıları bir sonraki derste anlatılacak.

Alıştırma önerisi

- Örneğe bir `Okul` sınıfı ekleyin. Öğrencinin okuduğu, öğretmenin çalıştığı okul bilgisi tutulsun. Bu bir association'dır, parça-bütün ilişkisi yoktur.
- Kişi bilgisi yazdırılırken okul bilgisi de yazdırılsın.
- Slaytlardaki hasta-doktor-tedavi örneği de bu yapıya uyarlanabilir.

Ders sonrası notlar

- Hoca, `Person`'daki get/set metotları `protected` olsaydı main sınıfından erişilemeyeceğini söyledi. Bu örnekte `Hafta5` main sınıfı `Person` ile **aynı pakette** (`Hafta5`). Java'da `protected` üyelere aynı paketteki tüm sınıflar erişebilir. Erişim yalnızca **başka paketteki** ilişkisiz sınıflar için kapanır.
- Hoca kod yorumuna "child to parent conversion" yazdı (01:03:19) ve "küçük bir sınıftan büyük bir sınıfa conversion yapılamaz" dedi. `Student pp = new Person(...)` hatasının yönü tam tersidir: **üst sınıftan alt sınıfa** (parent to child). Alt sınıftan üst sınıfa dönüşüm (upcasting) serbesttir.
- `this.bakiyeGuncelle()`: Hoca metodun içinde `this.` ile çağrılınca "yine aynısı çalışacak" dedi (00:56:46). Metodun kendi içinde yazılırsa üst sınıftakini değil kendini çağırır, sonsuz özyineleme olur. Java ile denendi: `StackOverflowError`. Üst sınıftaki metot için `super.` gerekir.
- Hoca belirsiz yapıcı çağrısı için "undefined behavior tarzı" dedi. Java'da bu derleme hatasıdır: `reference to A is ambiguous` (Java ile denendi).
- `super(...)` ilk satır kuralı dersin yapıldığı sürümler içindir. JDK 25'te `super(...)` öncesinde başka deyim yazılabilir: Java ile denendi, derlendi. `-source 21` ile aynı kod hata veriyor.
- Hoca örnek veri girerken ID için `int` yerine `long` ve `short` denedi. Derste son hâlde `int ID`, `int yas`, `double bakiye` kullanıldı (kareden).
- Derste `Student`'ın `toString()` metodunda alan adı yanlışlıkla "TECRUBE" yazıldı ve çıktıda "TECRUBE=Bilgisayar" göründü. Hoca bunu "BOLUM" olarak düzeltti.
- Yukarıdaki sınıflar Java ile derlenip çalıştırıldı. Çıktı, derste görülen değerlerle uyumludur (ör. öğrenci bakiyesi 15.0'dan 14.5'e, öğretmen bakiyesi 15.0'dan 14.0'a düşüyor).
- Altyazıdaki "akbit" Akbil, "vakiye/baki" bakiye, "Kulina", "Nesneye Yelene bir şey olmuş" gibi parçalar NetBeans'teki sorunla ilgili anlaşılmayan cümlelerdir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.